



HoodBoy Entertainment

LNKUP LABEL MANAGEMENT SYSTEM

TECHNICAL & OPERATIONAL GUIDE

LabelGrid API Integration Manual

A comprehensive step-by-step guide explaining how the HoodBoy Entertainment system operates after integrating the LabelGrid OpenAPI standard for automated digital music distribution, asset ingestion, status tracking, and webhooks.

1. SETUP &
ENV



2. ADMIN
HEALTH



3. ASSET
INGESTION



4. VALIDATION &
SUBMIT



5. WEBHOOKS &
SYNC

1 Architecture & System Overview

The HoodBoy Entertainment (HBE) Label Management System uses a dual-layer architecture combining local Mongoose/MongoDB persistence with LabelGrid's remote OpenAPI endpoints.



Key Architectural Principles

- **Single API Proxy:** The client browser communicates *only* with the HBE backend. It never calls LabelGrid endpoints directly.
- **Zero Secret Exposure:** Bearer tokens and webhook secrets reside strictly on the server side (``backend/.env``). No secrets exist in ``frontend/.env`` or client memory.
- **Idempotent Remote Linking:** Local MongoDB records store LabelGrid entity IDs (``labelgridArtistId``, ``labelgridReleaseId``, ``labelgridTrackId``). Retries always use existing IDs rather than creating duplicate remote entities.
- **Authoritative Remote Validation:** Distribution triggers local verification first, followed by LabelGrid's pre-flight validation API before final delivery.

LabelGrid OpenAPI Alignment

- **Documented Spec:** Built against the official public LabelGrid API standard (``https://api.labelgrid.com/docs/api``).
- **No Mock Fallbacks:** Operates strictly against real API endpoints in both sandbox and production environments.
- **Presigned S3 Assets:** Uses LabelGrid's presigned S3/cloud URL generation with local SHA-256 hash validation for reliable audio asset streaming.
- **HMAC Webhook Security:** Receives real-time state changes via cryptographically signed webhooks.

System Data Model & Linkage

HBE Local Model	LabelGrid Entity Mapping	Stored Identifiers	Synchronization Action
Artist (<code>`Artist.js`</code>)	LabelGrid Artist Profile	<code>`labelgridArtistId`</code> , <code>`labelgridSyncStatus`</code> , <code>`labelgridSyncError`</code>	Automated lookup/creation on submit or manual admin bulk sync.
Distribution Release (<code>`Distribution.js`</code>)	LabelGrid Release	<code>`labelgridReleaseId`</code> , <code>`labelgridStatus`</code> , <code>`labelgridValidationResult`</code>	Creation, cover image upload, DSP outlet target assignment, submit for distribution.
Embedded Track (<code>`Track`</code>)	LabelGrid Track	<code>`labelgridTrackId`</code> , <code>`labelgridUploadState`</code> , <code>`audioPresignedUrl`</code>	Presigned URL fetch, direct binary stream, SHA-256 confirmation.

Note on Legacy Data & Local Records:

Local MongoDB remains the authoritative source of truth for HBE label workflows, user access levels, and local financial ledgers. LabelGrid IDs are attached lazily without requiring destructive database migrations.

2 Environment Configuration & Setup

Proper configuration requires setting four environment variables on the backend server, whitelisting server IP addresses, and ensuring required node runtime capabilities.

1. Required Backend Environment Variables

Configure these variables inside backend/.env on your deployment server:

```
LABELGRID_API_TOKEN=your_long_lived_labelgrid_bearer_token
LABELGRID_BASE_URL=https://api.labelgrid.com/api/public
LABELGRID_LABEL_ID=10482
LABELGRID_WEBHOOK_SECRET=whsec_98f7a6b5c4d3e2f1a0b9c8d7e6f5a4b3
```

2. Server & Network Requirements

- **Node.js Runtime:** Node 20 LTS or higher required for native global fetch, FormData, Blob, and AbortSignal.timeout.
- **Outbound IP Allowlist:** Ensure your production/sandbox server static outbound IP is registered in the LabelGrid Developer Console.
- **Asset Persistence:** Keep backend/uploads/distribution directory persistent and readable across app restarts until asset confirmation completes.

3. Webhook Setup Instructions

- **Public Webhook URL:** Configure LabelGrid webhook endpoint to point to:
`https://api.hoodboyent.com/api/webhooks/labelgrid`
- **Subscribed Events:** Select release review, delivery, outlet status, transcode, and takedown events.
- **Secret Storage:** Copy the generated secret key into LABELGRID_WEBHOOK_SECRET.

Security Alert:

Never commit LABELGRID_API_TOKEN or LABELGRID_WEBHOOK_SECRET to git or pass them to frontend code. The backend uses static token headers for outgoing calls and HMAC signature validation for incoming webhooks.

System Initialization Flow

- 1 **Set Environment Variables:** Populate backend/.env with API credentials and restart the backend server daemon.
- 2 **Verify Admin Status:** Navigate to **Settings** → **Integrations** → **LabelGrid** in the Admin Dashboard.
- 3 **Perform Connection Health Test:** Click **Test Connection**. The system calls GET /me via backend service to confirm HTTP 200 connectivity.
- 4 **Execute Initial Artist Sync:** Click **Sync Artists** to match existing local HBE artists with LabelGrid profiles and store remote IDs.

3 Step-by-Step Operating Workflow (Steps 1 – 4)

Detailed operational breakdown of how music releases move from creation through media asset ingestion using the integrated LabelGrid API.

Step 1: Admin System Health & Bulk Synchronization

Admins begin in **Settings** → **Integrations** → **LabelGrid**. The backend executes background checks:

Indicator Status	Backend Condition	Admin Action Required
CONNECTED	HTTP 200 returned from `GET /me`. Token & Label ID valid.	Ready for operational release distribution.
NOT CONFIGURED	`LABELGRID_API_TOKEN` or `LABELGRID_LABEL_ID` missing.	Update `backend/.env` with valid credentials and restart backend.
AUTH ERROR	HTTP 401/403 from LabelGrid API.	Verify bearer token validity or IP allowlist settings in LabelGrid console.
UNAVAILABLE	Network timeout or HTTP 5xx error from LabelGrid server.	Check system logs and LabelGrid status page.

Step 2: Artist Profile Synchronization

Before submitting a release, artist metadata must be linked to LabelGrid. When an admin clicks **Sync Artists** or creates a new release:

- HBE checks if `artist.labelgridArtistId` exists.
- If missing, HBE queries LabelGrid by name/email or creates a new artist profile via `POST /api/public/artists`.
- The returned ID is saved into MongoDB to prevent duplicate profile creation.

Step 3: Release Creation & Dynamic Reference Data

- **Reference Data Fetch:** HBE fetches dynamic genres, languages, and DSP outlets directly via `GET /api/distribution/reference-data`.
- **Draft Draft Assembly:** The user fills title, main/featured artists, release date, UPC/catalog number, and explicit content ratings.
- **Entity Mapping:** Local release receives `labelgridReleaseId` upon creation call to `POST /api/public/releases`.

Step 4: Media Asset Upload Engine

- **Cover Art:** Multipart upload sends 3000x3000px JPG/PNG directly to `POST /api/public/releases/:id/artwork`.
- **Audio Presigned URL:** HBE requests a presigned AWS S3 upload URL for each track.
- **Stream & Hash:** Audio file streams directly to presigned URL; HBE calculates SHA-256 checksum simultaneously.
- **Confirmation:** Call sent to LabelGrid to mark track audio upload as complete.

Resiliency Mechanism:

If artwork or audio upload fails mid-flight, retrying the operation re-uses the existing `labelgridReleaseId` and `labelgridTrackId`, skipping redundant remote creation calls.

4 Step-by-Step Operating Workflow (Steps 5 – 8)

Finalizing distribution submission, automated validation, webhook tracking, and financial/analytics synchronization.

Step 5: Automated Validation & Distribution Submission

When the user clicks **Distribute Release**, HBE initiates a single-flight execution pipeline:

- 1 Locking UI:** The submission button is instantly disabled to prevent double-submissions.
- 2 Local Pre-check:** Validates presence of cover art, audio track files, ISRC codes, and legal metadata.
- 3 LabelGrid Validation API:** Calls LabelGrid validation endpoint (GET /api/public/releases/:id/validate).
- 4 Conditional Submission:** Distribution is triggered *only if* validation response is OK. If errors are returned, they are formatted and presented directly to the user.

Step 6: Delivery & Status Synchronization

Release status in HBE automatically updates based on normalized provider states:

LabelGrid Raw Status	HBE Display Status
`DRAFT` / `PENDING`	DRAFT
`VALIDATING`	VALIDATING
`PROCESSING` / `IN_REVIEW`	IN REVIEW
`DELIVERED` / `APPROVED`	DELIVERED
`REJECTED` / `NEEDS_ACTION`	ACTION REQ

Step 7: Automated Webhooks Engine

LabelGrid posts real-time event updates to /api/webhooks/labelgrid:

- **HMAC Signature Verification:** Backend verifies X-LabelGrid-Signature using raw request body and LABELGRID_WEBHOOK_SECRET.
- **Freshness Check:** Timestamps older than 5 minutes are rejected to block replay attacks.
- **Automatic Updates:** Updates delivery status across DSP outlets (Spotify, Apple Music, Tidal) automatically.

Step 8: Financial Analytics & Royalty Integration

LabelGrid royalty breakdown and streaming analytics are proxied via GET /api/distribution/labelgrid/royalties and GET /api/distribution/labelgrid/analytics. HBE's internal financial engine maintains local ledgers while embedding LabelGrid analytics for complete transparency.

5 Error Recovery Matrix & Admin Checklist

Troubleshooting guidelines, security safeguards, and standard operating procedures for label administrators.

Error Translation & Troubleshooting Matrix

Failure Scenario	System Diagnostic Message	Root Cause & Resolution Action
Validation Failure	`Validation Error: Missing Track Composer`	LabelGrid validation rejected release metadata. Open track metadata editor, populate writer/composer credits, and re-submit.
Audio Presigned Expiry	`Audio Upload Failed: Expired URL`	S3 presigned URL timed out. Click Retry Upload ; HBE will request a fresh presigned URL without duplicating the release.
Invalid Webhook Signature	`HTTP 401 Unauthorized: Invalid HMAC`	Webhook secret mismatch. Verify `LABELGRID_WEBHOOK_SECRET` in `backend/.env` matches LabelGrid developer console setting.
Rate Limit Exceeded	`HTTP 429 Too Many Requests`	LabelGrid API rate limit reached. HBE automatically enforces exponential backoff retries. No user action required.

Master Operator & Administrator Deployment Checklist

- 1 Environment Verification:** Confirm LABELGRID_API_TOKEN, LABELGRID_BASE_URL, LABELGRID_LABEL_ID, and LABELGRID_WEBHOOK_SECRET are loaded in backend environment.
- 2 Connection Health Test:** Open **Settings → Integrations → LabelGrid** and confirm green **CONNECTED** badge.
- 3 Artist Linkage:** Perform initial bulk sync to attach labelgridArtistId to all active label roster artists.
- 4 Artwork & Audio Preparation:** Confirm artwork is RGB 3000x3000px JPG/PNG and audio files are 16-bit 44.1kHz WAV.
- 5 Pre-Submission Verification:** Review ISRC codes, explicit tags, dynamic genres, and DSP target selections.
- 6 Submit & Monitor:** Click **Distribute Release**. Verify single-flight button locking and monitor status updates via dashboard or webhooks.

Crucial Safety Rule:

Do not alter local Mongoose labelgridReleaseId fields manually in MongoDB. Removing these IDs will cause retries to create duplicate remote releases on LabelGrid.